실습 손필기 노트

시험장에 가져갈 손필기 노트 초안이다. IntelliJ가 자동완성하지 않는 것만 시험 때 작업 순서대로 적고, ★가 많을수록 꼭 적는다. getter/setter(`Alt + Insert`)와 import(`Alt + Enter`)는 적지 않는다.

0장 — 시험 순서: A반 실제 출제 기준

A반 시험(10/26)은 sakila DB의 film 목록을 MVC로 출력하고, 화면 6장을 캡처해 PPT로 제출했다. 우리 반도 같은 틀에 DB·테이블 이름만 바뀔 가능성이 높다. 위에서부터 따라 하고, **캡처 ①~⑥** 표시가 나오면 그 화면을 찍어 둔다. 코드의 `// 자동` 줄은 IntelliJ가 만들어 주므로 적지 않는다. 1~3장은 막힐 때 찾아보는 참고용이다.

시험지에서 바꿔 끼울 것 ★★★

시험지의 이름을 그대로 옮겨 쓴다. 아래 코드는 우리 반 예상문제(world·city) 기준이다.

항목	우리 반 예상 (704)	A반 실제 (10/26)	바꾸는 곳
DB	`world`	`sakila`	JDBC URL 끝, Database 창
테이블	`city`	`film`	SQL의 `from`
컨트롤러	`controller/CityController`, `/world/cities`	`controller/FilmController`, `/sakila/films`	클래스 이름, `@WebServlet`
모델	`model/cityDTO`	`model/filmDTO`	클래스 이름, 필드 = 테이블 열
뷰	`webapp/world/cities.jsp`	`webapp/sakila/films.jsp`	폴더·파일, forward 경로
속성 이름	`cities`	`films`	`setAttribute` ↔ `items`

- 클래스 이름도 시험지 표기 그대로 쓴다. A반은 `filmDTO`를 소문자 그대로 만들었다.
- 열 이름: city는 `ID`, `Name`, `CountryCode`, `District`, `Population` / film은 `film_id`, `title`, `description`, `release_year` (A반 SQL 기준)

1단계 — 프로젝트 만들고 실행 확인

1. New Project → Jakarta EE: Name `jw학번`, Location `D:\jwb학번`, Template Web application, Application server Tomcat, Build system Gradle, Group `idusw.jwb학번` → Next → Create. 폴더·그룹에 반 표기를 붙인다(A반은 `jwa`).
2. Edit Configurations → Deployment → Application context를 `/jw학번`으로 바꾼다. A반은 기본값 그대로 둬서 주소가 `/Gradle___...war`처럼 길어졌다.
3. ▶ 실행 → 브라우저에 기본 index.jsp의 "Hello World!"가 뜨면 환경 OK

2단계 — build.gradle 의존성

dependencies 안에 4줄을 추가하고 `Ctrl + Shift + O`로 다시 로딩한다. A반은 mysql-connector-j 8.4.0을 썼다(9.7.0도 된다). 이 화면은 파일을 다 만든 뒤 프로젝트 트리와 함께 찍는다 → **캡처 ①**

```
implementation("jakarta.servlet.jsp:jakarta.servlet.jsp-api:4.0.0")
implementation("jakarta.servlet.jsp.jstl:jakarta.servlet.jsp.jstl-api:3.0.2")
implementation("org.glassfish.web:jakarta.servlet.jsp.jstl:3.0.1")
implementation("com.mysql:mysql-connector-j:8.4.0")
```

3단계 — DB 연결 확인

Database 창 + → Data Source → MySQL → Host `210.102.226.242`(241~244 중 하나), Port `8080`, root / cometrue → Test Connection → `N of M`을 눌러 시험지의 DB(world) 체크 → 테이블 열 이름 확인. 설정 창과 Database 창의 DB 트리가 함께 보이게 찍는다 → **캡처 ③**

4단계 — 패키지·폴더 만들기

- `java` 우클릭 → New → Package: `controller`, `model`. 기본 패키지 안이 아니라 `java` 바로 아래에 만든다. 시험지 경로와 같고 A반도 이렇게 했다. 기본 패키지의 HelloServlet은 그대로 둔다.
- `webapp` 우클릭 → New → Directory: 시험지의 뷰 폴더(`world`)
- 클래스는 New → Java Class, JSP는 New → JSP/JSPX로 만든다. JSP 맨 윗줄(page 지시자)과 html 뼈대는 자동으로 생긴다.

흐름 핵심: 네 군데 이름 맞추기 ★★★

폼 → 서블릿 → `setAttribute` → forward → 뷰(EL)는 아래 네 곳의 이름이 서로 맞아야 동작한다. 아래 5~8단계와 추가 대비 문제 모두 같은 원리다.

;연결	앞쪽	뒤쪽
폼 → 서블릿	`<form action="hello">`	`@WebServlet("/sample/hello")`

;연결	앞쪽	뒤쪽
입력칸 → 서블릿	`<input name="name">`	`request.getParameter("name")`
서블릿 → 뷰	`request.setAttribute("msg", ...)`	`${msg}`
forward → 파일	`getRequestDispatcher("/sample/hello-view.jsp")`	`webapp/sample/hello-view.jsp`

`action="hello"` 는 상대 경로라서, 폼 파일이 `sample` 폴더에 있으면 `/sample/hello` 로 간다.

안 될 때 이 표에서 찾는 법:

- 404 → 폼의 `action` 과 `@WebServlet` 주소, 또는 forward 경로와 실제 파일 위치
- 405 → 폼의 `method` 와 `doGet` · `doPost` 가 안 맞음
- null이나 빈칸 → `name` 과 `getParameter` , 또는 `setAttribute` 와 `${ }` 의 이름
- 한글 깨짐 → `doPost` 첫 줄에 `request.setCharacterEncoding("UTF-8")`

IntelliJ 단축키: 서블릿 안에서 `Ctrl + O` (Override Methods) → `doGet` · `doPost` 를 고르면 긴 선언이 자동으로 들어간다. 실행 중 코드를 고쳤으면 `Ctrl + F10` → Update classes and resources, 서블릿을 새로 만들었으면 Restart server.

5단계 — 모델: `model/cityDTO` → 캡처 ④

필드는 테이블 열과 같게 만든다. getter·setter는 `Alt + Insert` → Getter and Setter로 만든다. EL은 getter로 값을 읽으므로 반드시 만든다.

```
package model;                                          // 자동

public class cityDTO {                                  // 시험지 표기 그대로
    private int id;
    private String name;
    private String countryCode;
    private String district;
    private int population;
    // Alt + Insert → Getter and Setter
}
```

EL 이름은 getter에서 `get` 을 빼고 첫 글자를 소문자로 바꾼 것이다. `getCountryCode()` → `${city.countryCode}` . A반처럼 필드를 `film_id` 로 만들면 `getFilm_id()` → `${f.film_id}` 다. `${f.Film_id}` 가 아니다.

6단계 — 컨트롤러: `controller/CityController` → 캡처 ②

import는 `Alt + Enter`로 채우고 `java.sql` 쪽을 고른다. `doGet` 선언은 `Ctrl + O`로 넣는다.

```
package controller;                                  // 자동
import model.cityDTO;                                // 자동 (Alt + Enter)

@WebServlet(urlPatterns = "/world/cities")
public class CityController extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        String url = "jdbc:mysql://210.102.226.242:8080/world?
serverTimezone=UTC&useSSL=false";
        ArrayList<cityDTO> cities = new ArrayList<>();
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection(url, "root",
"cometrue");
            PreparedStatement pstmt = conn.prepareStatement("select * from
city limit 20");
            ResultSet rs = pstmt.executeQuery();
            while (rs.next()) {
                cityDTO city = new cityDTO();             // ① 객체를 먼저 만들
고
                city.setId(rs.getInt("ID"));              // ② setter는 한 줄
씩
                city.setName(rs.getString("Name"));
                city.setCountryCode(rs.getString("CountryCode"));
                city.setDistrict(rs.getString("District"));
                city.setPopulation(rs.getInt("Population"));
                cities.add(city);                         // ③ 리스트에 담기
            }
            rs.close(); pstmt.close(); conn.close();
        } catch (Exception e) {
            e.printStackTrace();                          // 오류는 아래 콘솔에
출력
        }
        request.setAttribute("cities", cities);
        request.getRequestDispatcher("/world/cities.jsp").forward(request,
response);
    }
}
```

A반 답안에서 나온 실수:

- `new filmDTO(f.setFilm_id(...), ...)` 처럼 setter를 생성자 괄호 안에 넣으면 컴파일 오류다. 객체를 만든 뒤 setter를 한 줄씩 호출한다.
- `list.add()` 를 빠뜨리면 표가 비어 나온다.
- `forward("film.jsp")` 처럼 실제 파일 이름(`films.jsp`)과 다르면 화면이 안 나온다. `/` 부터 쓰는 절대 경로로 파일 이름을 정확히 맞춘다.
- `limit 20` 은 결과 화면을 짧게 하는 요령이다. 전체 출력을 요구하면 뺀다.

7단계 — 뷰: `webapp/world/cities.jsp` → 캡처 ⑤

JSP 템플릿이 page 지시자와 html 뼈대를 만들어 준다. 스크립틀릿 버전으로 동작을 확인한 뒤 JSTL&EL로 바꾸고, 제출은 JSTL&EL 버전으로 한다.

```
<%-- 변환 전: 스크립틀릿·표현식 --%>
<%@ page import="model.cityDTO, java.util.ArrayList" %>
<table border="1">
    <tr><th>이름</th><th>주</th><th>인구수</th></tr>
<%
    ArrayList<cityDTO> cities = (ArrayList<cityDTO>)
request.getAttribute("cities");
    for (cityDTO city : cities) {
%>
    <tr><td><%= city.getName() %></td><td><%= city.getDistrict() %></td><td>
<%= city.getPopulation() %></td></tr>
<% } %>
</table>
```

```
<%-- 변환 후: JSTL & EL (제출할 답) --%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<table border="1">
    <tr><th>이름</th><th>주</th><th>인구수</th></tr>
    <c:forEach var="city" items="${cities}">
        <tr><td>${city.name}</td><td>${city.district}</td>
<td>${city.population}</td></tr>
    </c:forEach>
</table>
```

`<c:forEach>` 에는 `items` (서블릿의 `setAttribute` 이름)와 `var` 가 꼭 있어야 한다. A반 답안은 둘 다 빠져서 반복이 실행되지 않았다.

8단계 — 실행 확인 → 캡처 ⑥

1. 서블릿을 새로 만들었으므로 Restart server 또는 `Shift + F10` 으로 실행한다.
2. 브라우저에서 **컨트롤러 주소**로 접속한다: `localhost:8080/jw학번/world/cities` .
   `cities.jsp` 를 직접 열면 컨트롤러를 거치지 않아 표 제목만 나온다. A반 결과 화면이 이랬다.

3. 도시 표가 나오면 주소창이 보이게 찍는다 → **캡처 ⑥**

안 될 때: 프로젝트 트리에서 빨간 물결줄이 있는 폴더를 따라 내려가면 컴파일 오류가 난 파일이 나온다. 표가 비면 아래 콘솔의 오류 메시지를 보고 1장 ④ 표에서 찾는다.

9단계 — PPT로 제출

A반 제출 파일은 글 없이 전체 화면 캡처 6장이었다. 이 순서로 슬라이드 한 장에 하나씩 붙인다.

슬라이드	캡처할 화면	찍는 때
①	프로젝트 트리 + build.gradle	모든 파일을 만든 뒤
②	컨트롤러 코드	6단계 후
③	DB 연결 설정 창 + Database 창의 DB 트리	3단계
④	모델(DTO) 코드	5단계 후
⑤	뷰 JSP 코드	7단계 후
⑥	브라우저 결과 (주소창 포함)	8단계

- 파일 이름: `jw학번_이름.pptx` (A반 예: `jw202312017_최민수.pptx`)
- 캡처: `Win + Shift + S`, 전체 화면은 `PrtSc`
- 제출 전 Tomcat을 중지한다(`Ctrl + F2`). 프로젝트 압축이나 PDF 변환 같은 추가 지시가 있으면 따른다.

추가 대비 A — 입력값으로 검색 ★★

입력, DB 조회, 뷰 전달이 합쳐진 형태다. 시험지에 "입력한 값으로 조회" 조건이 붙으면 이렇게 바꾼다.

`webapp/world/search.jsp` 를 New → JSP/JSPX로 만들고 `<body>` 안에 적는다. 이 파일이 `world` 폴더에 있어서 `action="cities"` 는 `/world/cities` 로 간다.

```
<form action="cities" method="get">
    국가 코드: <input type="text" name="code"> <input type="submit" value="검색">
</form>
```

6단계 컨트롤러에서 `prepareStatement` 와 `executeQuery` 두 줄을 아래로 바꾼다. 입력이 없으면 전체를 조회하므로 `/world/cities` 도 그대로 동작한다.

```
String code = request.getParameter("code");
PreparedStatement pstmt;
if (code == null || code.isEmpty()) {
    pstmt = conn.prepareStatement("select * from city limit 20");
// 입력 없으면 6단계와 같음
} else {
    pstmt = conn.prepareStatement("select * from city where countrycode =
?");
    pstmt.setString(1, code);
// ? 순서는 1부터
}
ResultSet rs = pstmt.executeQuery();
// while (rs.next()) 부분과 close는 6단계와 같다
```

뷰는 7단계의 `cities.jsp`를 그대로 쓴다. 표 위에 `<p>검색: ${param.code}</p>`를 넣으면 입력값도 EL로 보여 줄 수 있다.

- 폼을 `method="post"`로 내라는 조건이면, CityController에 doPost를 만들고 안에서 `doGet(request, response);`를 호출한다.

확인: `/<context>/world/search.jsp`에서 `KOR`를 입력하면 한국 도시만 나온다.

추가 대비 B — 구구단 ☆

A반 시험에는 나오지 않았고 이미 과제로 한 내용이다. 핵심은 B-2 폼, 서블릿의 ★ 줄(입력 받기 → Model에 담기 → setAttribute → forward), B-4 뷰 변환이다. ☆ 사이의 구구단 계산은 시간이 남을 때 본다.

B-1. `model/GugudanDTO.java`

```
package model;                          // 자동
import java.util.ArrayList;                             // 자동 (Alt + Enter)

public class GugudanDTO {
    private ArrayList<String> stringArrayList = new ArrayList<>();
    // Alt + Insert → Getter and Setter
}
```

B-2. `webapp/sample/gugudan-form.jsp` — JSP 템플릿의 `<body>` 안에 적는다.

```
<form action="./gugudan.do" method="post">
    구구단 출력 - Using Columns : <input type="text" name="cols">
    <input type="submit" value="출력">
</form>
```

B-3. `controller/GugudanServlet.java` — import는 `Alt + Enter`로 채운다.

```
@WebServlet(urlPatterns = "/sample/gugudan.do")
public class GugudanServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        int cols = Integer.parseInt(request.getParameter("cols"));   // ★ 입력
받기
        // ☆ 구구단 계산 시작 ──────────────
        int rows = (8 + cols - 1) / cols;                // 2~9단을 cols개씩,
올림
        ArrayList<String> result = new ArrayList<>();
        result.add("<table>");
        for (int i = 1; i <= rows; i++) {
            int start = 2 + (i - 1) * cols;              // 이번 묶음의 시작 단
            int end = Math.min(start + cols, 10);        // 9단까지
            for (int mul = 1; mul <= 9; mul++) {
                result.add("<tr>");
                for (int dan = start; dan < end; dan++)
                    result.add("<td width=100px>" + dan + " * " + mul + " = "
+ dan * mul + "</td>");
                result.add("</tr>");
            }
            result.add("<tr><td> </td></tr>");      // 묶음 사이 빈 줄
        }
        result.add("</table>");
        // ☆ 구구단 계산 끝 ──────────────
        GugudanDTO dto = new GugudanDTO();                // ★ Model에 담기
        dto.setStringArrayList(result);
        request.setAttribute("dto", dto);                 // ★ 뷰로 넘길 속성
        request.getRequestDispatcher("/sample/gugudan-
view.jsp").forward(request, response);   // ★ forward
    }
}
```

B-4. **`webapp/sample/gugudan-view.jsp`** — 스크립틀릿 버전으로 먼저 동작을 확인한 뒤 JSTL&EL로 바꾼다.

```
<%-- 변환 전: 스크립틀릿·표현식 --%>
<%@ page import="model.GugudanDTO" %>
<form action="gugudan.do" method="post">
    구구단 출력 - Using Columns : <input type="text" name="cols" value="<%=
request.getParameter("cols") %>">
    <input type="submit" value="출력">
</form>
<%
    GugudanDTO dto = (GugudanDTO) request.getAttribute("dto");
    for (String str : dto.getStringArrayList()) out.println(str);
%>
```

```
<%-- 변환 후: JSTL & EL (제출할 답) --%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<form action="gugudan.do" method="post">
    구구단 출력 - Using Columns : <input type="text" name="cols"
value="${param.cols}">
    <input type="submit" value="출력">
</form>
<c:forEach var="str" items="${dto.stringArrayList}">${str}</c:forEach>
```

확인: `/<context>/sample/gugudan-form.jsp`에서 3을 입력하면 2~4, 5~7, 8~9단 세 묶음이 나온다.

1장 — 설정, DB, 오류

① 프로젝트 생성·실행 ★★

- New Project → **Jakarta EE**: Name `jw학번`, Template **Web application**, Build system **Gradle**, Application server **Tomcat 11**, JDK 25.
- Group은 기본 패키지가 된다(예: `idusw.jwb학번`). 시험의 `controller`·`model`은 `java` 바로 아래에 따로 만든다(0장 4단계).
- Run 설정: Edit Configurations → Deployment → `war exploded`, Application context 확인. 테스트 주소는 `localhost:8080/<context>/world/cities`.
- 플러그인 확인(`Ctrl + Alt + S` → Plugins): Tomcat and TomEE, Jakarta EE: Server Pages (JSP), Database Tools and SQL.
- 단축키: Gradle 다시 로딩 `Ctrl + Shift + O`, 실행 `Shift + F10`, 중지 `Ctrl + F2`.

①-1 프로젝트 구성 ★★

프로젝트를 만들자마자 이 구조대로 패키지와 폴더를 만든다.

```
jw학번/
├─ build.gradle                          ← 의존성 4줄 추가
└─ src/main/
   ├─ java/
   |   ├─ controller/
   |   |   ├─ CityController.java        ← /world/cities
   |   |   └─ GugudanServlet.java        ← ☆ /sample/gugudan.do
   |   ├─ model/
   |   |   ├─ cityDTO.java               ← 시험지 표기 그대로
   |   |   └─ GugudanDTO.java            ← ☆
   |   └─ idusw.jwb학번.jw학번/           ← 자동 생성 (HelloServlet), 그대로 둠
   └─ webapp/
       ├─ world/
       |   └─ cities.jsp                 ← 도시 목록, JSTL&EL
       ├─ sample/                        ← ☆ gugudan-form.jsp, gugudan-view.jsp
       └─ index.jsp                      ← 자동 생성
```

구조도 기호 읽는 법:

기호	뜻
`├─`	위 폴더 안에 든 항목. 같은 폴더에 아래로 항목이 더 있다
`└─`	그 폴더의 마지막 항목
`|`	위 폴더의 목록이 아래로 계속 이어진다는 세로선
들여쓰기	한 칸 들어갈 때마다 한 단계 안쪽 폴더
이름 끝의 `/`	폴더. `.java`, `.jsp`로 끝나면 파일
`←` 뒤의 글	설명 메모. 파일 이름이 아니다

- `controller`·`model`은 `java` 바로 아래에 둔다. 파일 첫 줄은 `package controller;`, `package model;`이고, 다른 패키지의 클래스는 `import model.cityDTO;`처럼 불러온다.
- 프로젝트 트리에서 폴더에 빨간 물결줄이 생기면 그 안의 파일에 컴파일 오류가 있다는 뜻이다.
- ☆ 구구단 컨트롤러 URL은 `/sample/gugudan.do`다. 폼이 `sample` 폴더에 있어서 `action="./gugudan.do"`가 이 주소로 간다.

② build.gradle 추가 줄 ★★★

새 프로젝트에는 `servlet-api` 한 줄만 있다. 아래 세 줄을 dependencies에 추가하고 다시 로딩한다.

```
implementation 'jakarta.servlet.jsp.jstl:jakarta.servlet.jsp.jstl-api:3.0.2'
implementation 'org.glassfish.web:jakarta.servlet.jsp.jstl:3.0.1'
implementation 'com.mysql:mysql-connector-j:9.7.0'
```

③ MySQL 연결 ★★★

IntelliJ Database 창: + → Data Source → MySQL → Host `210.102.226.24x`, Port `8080`, root / cometrue → Test Connection → `N of M` 표시를 눌러 world 체크.

```
Class.forName("com.mysql.cj.jdbc.Driver");
"jdbc:mysql://210.102.226.242:8080/world?serverTimezone=UTC&useSSL=false"
```

`city` 테이블 열: `ID` (int), `Name`, `CountryCode`, `District`, `Population` (int)

④ 오류 → 원인 ★★

오류	원인과 해결
Required plugin ... is disabled	Fix 버튼 → IntelliJ 재시작
package ... does not exist	package 선언, 폴더 위치, import 불일치
unclosed string literal	따옴표 누락 또는 파일이 잘림
Access denied (using password: NO)	비밀번호 미입력
Table 'mysql.city' doesn't exist	URL 끝을 `/world`로
404	URL, context, JSP 위치가 어긋남
405	GET으로 접근했는데 `doGet` 없음
500 + NullPointerException	JSP를 주소창으로 직접 열어 forward를 안 거침
EL이 빈칸	items·var 이름 불일치, getter 없음, `${requestScope.city}` 사용
포트 사용 중	Tomcat HTTP port(8080) 또는 JMX port(1099) 변경
입력한 한글이 깨짐	doPost 첫 줄에 `request.setCharacterEncoding("UTF-8")`, JSP는 page 지시자에 charset=UTF-8
CSS·이미지가 안 나옴	상대 경로의 기준은 주소창 URL의 폴더. 컨트롤러 URL을 JSP와 같은 폴더로 두거나 `${pageContext.request.contextPath}/...` 사용

오류	원인과 해결
redirect 후 값이 빈칸	sendRedirect는 새 요청이라 request 속성이 사라짐 → forward를 쓰거나 session에 저장
폴더에 빨간 물결줄	그 안의 파일에 컴파일 오류. 물결줄을 따라 내려가 찾는다
표 제목만 나오고 데이터가 없음	JSP를 직접 열었음 → 컨트롤러 URL로 접속. 또는 list.add() 누락
주소가 /Gradle___...war처럼 김	Application context를 /jw학번으로 바꾸지 않음

2장 — 코드

전체 코드는 0장 5~7단계와 추가 대비 B에 있다. 이 장은 외울 핵심 줄만 모은 요약이므로, 손필기는 0장을 우선한다.

⑤ 구구단 MVC ★★

form 핵심 두 줄: `<form action="./gugudan.do" method="post">`, `<input type="text" name="cols">`. 서블릿은 `@WebServlet("/sample/gugudan.do")` 이고, 가장 외우기 어려운 것은 아래 공식이다.

```
int cols = Integer.parseInt(request.getParameter("cols"));
int rows = (8 + cols - 1) / cols;            // 2~9단을 cols개씩, 올림
start = 2 + (i - 1) * cols;                   // i번째 행의 시작 단
end = Math.min(start + cols, 10);             // dan < end, 9단까지
request.setAttribute("dto", dto);
request.getRequestDispatcher("/sample/gugudan-view.jsp").forward(request,
response);
```

⑥ CityController 뼈대 ★★★

```
@WebServlet(name = "cityController", urlPatterns = {"/world/cities"})
// doGet → process(request, response)
Connection conn = DriverManager.getConnection(URL, "root", "cometrue");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("select * from city");
while (rs.next()) {
    CityDTO city = new CityDTO();
    city.setName(rs.getString("name"));   // district, population도 같은 방식
    list.add(city);
}
request.setAttribute("cities", list);
request.getRequestDispatcher("/world/cities.jsp").forward(request, response);
// catch (SQLException e), finally에서 rs → stmt → conn 순서로 close
```

CityDTO는 필드만 적는다: `int id`, `String name`, `countryCode`, `district`, `int population`. getter/setter는 `Alt + Insert`로 만든다. EL은 getter로 값을 읽으므로 getter는 반드시 있어야 한다.

⑦ 뷰 변환: 스크립틀릿 → JSTL&EL ★★★

시험에서 평가하는 핵심 역량이라 양쪽 버전을 모두 적는다.

```
<%-- 스크립틀릿 --%>
<%@ page import="...model.CityDTO, java.util.ArrayList" %>
<% ArrayList<CityDTO> cities = (ArrayList<CityDTO>)
request.getAttribute("cities");
   for (CityDTO dto : cities) { %>
   <tr><td><%= dto.getName() %></td> ... </tr>
<% } %>

<%-- JSTL&EL --%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<c:forEach var="city" items="${cities}">
   <tr><td>${city.name}</td><td>${city.district}</td><td>${city.population}
</td></tr>
</c:forEach>
```

변환 규칙:

- `getAttribute("x")` → `${x}`
- `getXxx()` → `.xxx`
- `getParameter("p")` → `${param.p}`
- `for` → `c:forEach`, `if` → `c:if` / `c:choose`

구구단 view의 JSTL 버전: `value="${param.cols}"`, `<c:forEach var="str" items="${dto.stringArrayList}">${str}</c:forEach>`. 함정: `str`에 HTML 태그가 들어 있어 `<c:out>`을 쓰면 표가 깨진다.

⑧ 제출 절차 ☆

0장 9단계의 캡처 6장 순서를 따른다. 작년에는 PDF와 프로젝트 폴더를 함께 압축해 냈다는 말도 있으니, 당일 제출 안내를 확인한다.

참고

- A반은 시험지 표기 그대로 `filmDTO`(소문자)로 만들었다. 우리도 `cityDTO`로 나오면 그대로 쓴다. 자바 관례와 달라 IntelliJ가 경고를 띄워도 오류는 아니다.
- 노트를 다 만든 뒤 노트북에서 빈 프로젝트부터 이 노트만 보고 시간을 재며 한 번 끝까지 해 본다. 막힌 지점을 노트에 추가한다.

3장 — 작년 기출·다른 반 출제 대비

작년 중간고사(sb-admin-2 템플릿 + 회원 등록 실습)와 오늘 다른 반 출제(폼 입력 → 뷰 전달, 프로젝트 생성 + SQL 연결)를 반영한 추가 메모다. 올해는 작년 흐름에 DB 연결과 JSTL·EL이 더해진 형태로 예상한다.

⑨ 입력 → 컨트롤러 → 뷰 전달 ★★★

작년 등록 실습과 오늘 다른 반 출제의 공통 뼈대다. 구구단과 같은 흐름이고, 값만 다르다.

```
<%-- register.jsp: 입력 폼 (주소가 /sba2/register이므로 action="save"는
/sba2/save) --%>
<form action="save" method="post">
    <input type="text" name="fname"> <input type="text" name="lname">
    <input type="email" name="email"> <input type="password" name="pw">
    <input type="submit" value="등록">
</form>
```

```
@WebServlet(urlPatterns = {"/sba2/register", "/sba2/save"})
// doPost
request.setCharacterEncoding("UTF-8");                  // 한글 입력, 첫 줄에
request.setAttribute("name", request.getParameter("lname") +
request.getParameter("fname"));
request.setAttribute("email", request.getParameter("email"));
request.getRequestDispatcher("/sba2/blank.jsp").forward(request, response);
```

```
<%-- blank.jsp: 뷰, 두 가지 표기 --%>
${name} ${email}                              <%-- EL --%>
<%= request.getAttribute("name") %>           <%-- 표현식 --%>
```

폼의 `name` 속성 값이 곧 `getParameter()`의 이름이다. 하나라도 다르면 null이 된다.

⑩ forward vs sendRedirect ★★

구분	forward	sendRedirect
코드	`request.getRequestDispatcher("/sba2/blank.jsp").forward(request, response)`	`response.sendRedirect(request.getContextPath() + "/sba2/blank")`
요청	같은 요청을 서버 안에서 넘김	브라우저가 새 요청을 보냄
주소창	처음 URL 그대로 (`/save`)	새 URL로 바뀜 (`/blank`)
request 속성	유지됨 → `${name}` 출력됨	사라짐 → session이나 application에 저장해야 함
경로	컨텍스트 경로를 빼고 `/sba2/...`	`getContextPath()`를 앞에 붙임

작년 선배 답안은 sendRedirect를 쓰면서 `application.setAttribute()`로 값을 넘겼다. 동작은 하지만 application은 모든 사용자가 공유하므로, 사용자별 값은 `request.getSession().setAttribute()`가 더 알맞다. 뷰에서는 `${sessionScope.name}` 또는 `${name}`으로 읽는다.

스크립틀릿 ↔ EL 변환 예: `<%= (String) application.getAttribute("name") %>` → `${applicationScope.name}` 또는 `${name}`

⑪ 입력값으로 DB 조회·저장 ★★★

"입력 + SQL 연결"이 합쳐지면, 폼에서 받은 값을 SQL의 `?` 자리에 넣는 PreparedStatement가 나온다. 테이블·열 이름은 문제에 맞게 바꾼다.

```
// 입력값으로 조회: 예) 국가 코드로 도시 검색
String code = request.getParameter("code");
PreparedStatement pstmt = conn.prepareStatement("select * from city where
countrycode = ?");
pstmt.setString(1, code);                  // ? 순서는 1부터
ResultSet rs = pstmt.executeQuery();

// 입력값 저장: 예) 회원 등록
pstmt = conn.prepareStatement("insert into member(name, email, pw) values(?,
?, ?)");
pstmt.setString(1, name); pstmt.setString(2, email); pstmt.setString(3, pw);
int rows = pstmt.executeUpdate();          // 반영된 행 수
```

- select는 `executeQuery()` → ResultSet, insert·update·delete는 `executeUpdate()` → int.

- 숫자 열은 `setInt()`, 문자 열은 `setString()`을 쓴다.
- 입력값을 SQL 문자열에 직접 이어 붙이지 않는다. 따옴표 오류와 SQL 인젝션 위험이 있다.

⑫ 확장자 없는 URL: 컨트롤러 하나로 여러 화면 ★★

작년 문제 "모든 메뉴를 .jsp 없이 접근"의 답이다. URL 마지막 조각을 잘라 같은 이름의 JSP로 forward 한다.

```
@WebServlet(name = "homeController",
        urlPatterns = {"/sba2/index", "/sba2/buttons", "/sba2/register",
"/sba2/save"})
// doGet
String uri = request.getRequestURI();                        // /a-
2004/sba2/buttons
String action = uri.substring(uri.lastIndexOf('/') + 1);    // buttons
String view = "/sba2/" + action + ".jsp";                    //
/sba2/buttons.jsp
request.getRequestDispatcher(view).forward(request, response);
```

- 메뉴 링크도 함께 바꾼다: `href="buttons.html"` → `href="buttons"`.
- urlPatterns에 없는 URL은 404가 나므로, 확장자 없이 열 메뉴는 모두 등록한다.
- 컨트롤러 URL을 JSP와 같은 폴더(`/sba2/...`)에 두면 템플릿의 css·js 상대 경로가 그대로 맞는다.

⑬ 포트·컨텍스트·첫 화면 이동 ★★

작년에는 `http://localhost:8888/<반-학번>`으로 배포하라는 조건이 있었다.

- 포트: Edit Configurations → Server 탭 → HTTP port (예: `8888`)
- 컨텍스트: Deployment 탭 → Application context (예: `/a-2004`)
- Server 탭의 URL도 같은 주소로 맞추면 실행할 때 바로 열린다.
- 첫 화면 이동: webapp의 `index.jsp`에 한 줄을 넣는다.

```
<% response.sendRedirect(request.getContextPath() + "/sba2/index"); %>
```

⑭ HTML 템플릿을 JSP로 ☆

1. 템플릿(sb-admin-2) 압축을 `webapp/sba2`에 푼다.
2. 필요한 `.html` 파일을 우클릭 → Refactor → Rename(`Shift + F6`)으로 `.jsp`로 바꾼다.
3. JSP 맨 위에 page 지시자를 추가해 한글 깨짐을 막는다.

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
```